

分析并提升StyleGAN的图像质量

陶若 纳维达 萨穆尔 纳维达 米·艾塔拉 纳维达 扬·赫尔斯特
恩 纳维达 雅·蒂宁 纳维达 和 阿尔托大学 提·莫拉 纳维达

摘要

The style-based GAN architecture (StyleGAN) yields state-of-the-art results in data-driven unconditional generative image modeling. We expose and analyze several of its characteristic artifacts, and propose changes in both model architecture and training methods to address them. In particular, we redesign the generator normalization, revisit progressive growing, and regularize the generator to encourage good conditioning in the mapping from latent codes to images. In addition to improving image quality, this path length regularizer yields the additional benefit that the generator becomes significantly easier to invert. This makes it possible to reliably attribute a generated image to a particular network. We furthermore visualize how well the generator utilizes its output resolution, and identify a capacity problem, motivating us to train larger models for additional quality improvements. Overall, our improved model redefines the state of the art in unconditional image modeling, both in terms of existing distribution quality metrics as well as perceived image quality.

1. 引言

生成方法（尤其是生成对抗网络（GAN）[16]）所产生图像的分辨率和质量正在迅速提高[23, 31, 5]。目前最先进的高分辨率图像合成方法是StyleGAN [24]，它已被证明在多种数据集上可靠运行。我们的工作重点在于修复其特征性伪影并进一步提升结果质量。

StyleGAN[24]的一个显著特点是其非传统的生成器架构。它不是仅仅将输入潜在代码 $z \in \mathcal{Z}$ 馈送到网络的开始部分，而是先将其转换为中间潜在代码 $w \in \mathcal{W}$ 。接着，仿射变换产生控制通过自适应实例归一化（AdaIN）[21, 9, 13, 8]操作的各层的参数 $styles$ 。此外，通过提供随机变化来促进随机变异。

额外的随机噪声映射到合成网络。已有研究表明[24, 38]，这种设计使得中间潜在空间 $\mathcal{W}$ 比输入潜在空间 $\mathcal{Z}$ 更加松散耦合。在本文中，我们仅针对 $\mathcal{W}$ 进行所有分析，因为它从合成网络的角度来看相关的潜在空间。

许多观察者注意到，由StyleGAN [3]生成的图像中存在一些特征性伪影。我们识别了导致这些伪影的两个原因，并描述了通过改变架构和训练方法来消除这些问题的方法。首先，我们研究了常见块状伪影的起源，发现生成器创建这些伪影是为了规避其架构中的设计缺陷。在第2节中，我们将重新设计生成器使用的归一化方法，从而去除这些伪影。其次，我们分析了与渐进式增长[23]相关的伪影，该技术已被证明在稳定高分辨率GAN训练方面非常成功。我们提出了一种替代设计方案，在不改变网络拓扑的情况下实现相同目标——即从低分辨率图像开始训练，并逐步将重点转移到更高分辨率的图像上。这种新设计还使我们能够分析生成图像的有效分辨率，发现其实际有效分辨率低于预期，这促使我们需要增加模型容量（见第4节）。

对生成方法产生的图像质量进行定量分析仍然是一个具有挑战性的课题。Fréchet inception距离（FID）[20]通过测量两个分布在InceptionV3分类器的高维特征空间中的密度差异来评估这些分布之间的区别。Precision和Recall（P&R）[36, 27]则分别明确量化了与训练数据相似的生成图像的比例以及可以由模型生成的训练数据比例，从而提供了额外的视角。我们使用这些指标来衡量改进情况。

感知路径长度（PPL）指标最初作为估计潜在空间质量的一种方法被提出，我们发现该指标与分类器网络近期表现出的以关注纹理而非形状为特点的现象一致，并且由此导致这些度量标准无法准确捕捉图像质量的所有方面。



图1. 归一化实例会在StyleGAN图像中产生类似水滴的伪影。这些伪影并不总是在生成的图像中显而易见，但如果观察生成网络内部的激活情况，则在所有从64x64分辨率开始的特征图中都能发现这一问题，这是一个系统性的问题，困扰着所有的StyleGAN图像。

插值与形状的一致性和稳定性相关。基于此，我们在第3节中对合成网络进行正则化处理，以促进平滑映射，并显著提升了质量。为应对计算成本问题，我们还建议减少所有正则化的执行频率，观察到这并不会影响其有效性。

最终，我们发现将图像投影到潜在空间 $\mathcal{W}$ 中，在使用新的路径长度正则化的StyleGAN2生成器时表现显著优于原始的StyleGAN。这使得更容易将生成的图像归因于其来源（见第5节）。

我们的实现和训练模型可在 <https://github.com/NVlabs/stylegan2> 获取。

2. 消除规范化伪影

我们首先观察到，由StyleGAN生成的大多数图像都表现出类似水滴形状的特征性斑块状伪影。如图1所示，即使在最终图像中可能不易察觉该水滴，它仍然存在于生成器的所有中间特征图中。¹这种异常现象大约从64×64分辨率开始出现，并且随着分辨率的提高而逐渐增强。令人困惑的是，判别器本应能够检测到此类一致性的伪影。

我们将问题归因于AdaIN操作，该操作分别对每个特征图的均值和方差进行标准化，从而可能破坏了各特征之间幅度信息的相关性。我们假设水滴状伪影是生成器故意绕过实例规范化以传递信号强度信息的结果：通过创建一个强烈的局部峰值来主导统计量，生成器可以在其他地方随意调整信号强度。如以下详细说明，在移除生成器中的标准化步骤后，水滴状伪影完全消失，这一发现支持了我们的假设。

¹In rare cases (perhaps 0.1% of images) the droplet is missing, leading to severely corrupted images. See Appendix A for details.

2.1. 生成器架构再探

我们将首先对StyleGAN生成器的几个细节进行修改，以更好地适应我们重新设计的归一化方法。这些更改本身在质量指标上要么没有影响，要么有轻微的正面效果。

图2a展示了原始的StyleGAN合成网络 g [24]，而在图2b中，我们通过展示权重和偏置，并将AdaIN操作分解为其两个组成部分：归一化和调制，来详细展开该图。这使我们可以重新绘制概念上的灰色框，每个框表示一个风格活跃的网络部分（即“风格块”）。有趣的是，原始StyleGAN在风格块内应用了偏置和噪声，导致它们相对于当前风格幅度的影响呈反比关系。我们观察到，将这些操作移出风格块并在归一化数据上进行操作可以获得更可预测的结果。此外，我们注意到，在此更改之后，仅需对标准差进行归一化和调制（即不需要均值）。将偏置、噪声和归一化应用于常量输入的操作也可以安全地删除而不会产生明显的缺点。这种变体如图2c所示，并作为我们重新设计的归一化的起点。

2.2. 实例归一化再探

StyleGAN的一个主要优势是可以通过*style mixing*控制生成的图像，即在推理时向不同层输入不同的潜在变量 $\mathbf{w}$ 。实际上，风格调制可能会使某些特征图放大一个数量级以上。为了使风格混合正常工作，我们必须针对每个样本显式地抵消这种放大——否则后续层将无法以有意义的方式处理数据。

如果我们愿意牺牲特定规模的控制（参见视频），我们可以简单地移除归一化，从而消除这些伪影，并略微提高FID [27]。我们现在提出一个更好的替代方案，在去除伪影的同时保持完全可控性。主要思想是基于输入特征图的*expected*统计信息进行归一化，但不进行显式强制。

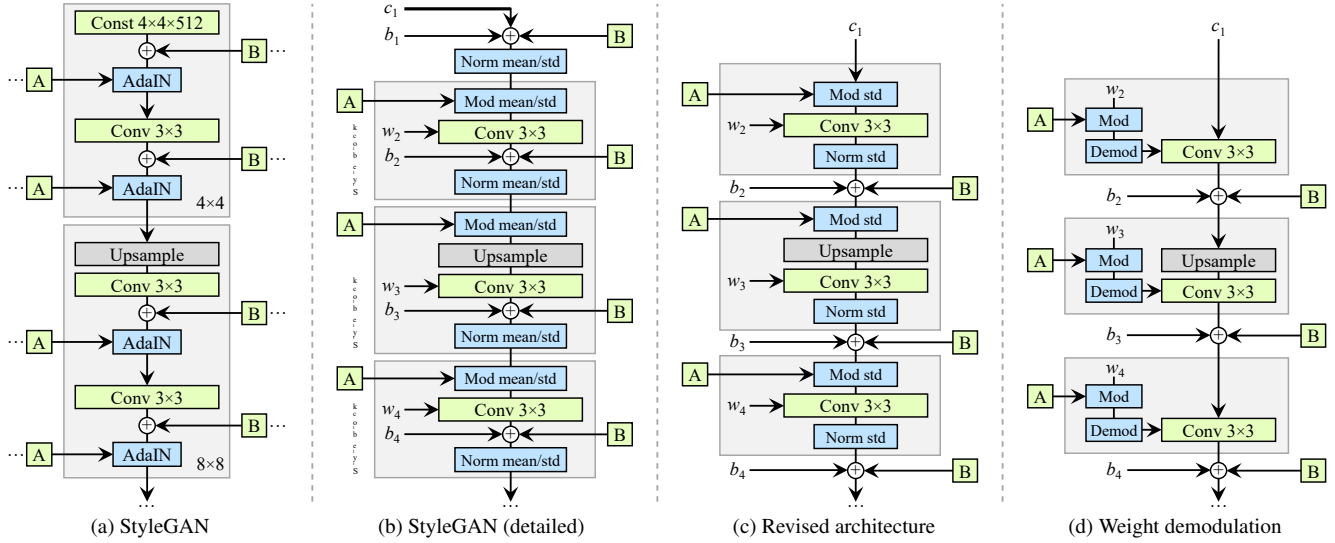


图2. 我们重新设计了StyleGAN合成网络的架构。(a) 原始的StyleGAN，其中A表示从 W 学习到的仿射变换 $\hat{w}$ 生成样式，B是噪声广播操作。(b) 带有完整细节的相同图表。在这里我们将AdaIN分解为显式的归一化后接调制，两者都针对每个特征图的均值和标准差进行操作。我们还标注了学习到的权重(w)、偏差(b)以及常量输入(c)，并重新绘制灰色框以使每个框中只有一个样式处于活动状态。激活函数(leaky ReLU)总是在添加偏差后立即应用。(c) 我们在原始架构的基础上进行了若干改动，这些改动在正文中得到了解释。我们移除了一些冗余的初始操作，将 b 和 B 的相加操作移到了每个样式的活跃区域之外，并仅调整每个特征图的标准差。(d) 修订后的架构使我们可以用“解调”操作替换实例归一化，我们将此操作应用于与每一卷积层相关的权重。

回想图2c中的风格块由调制、卷积和归一化组成。我们先考虑调制后接卷积的效果。调制根据输入的风格对卷积的每个输入特征图进行缩放，这也可以通过缩放卷积权重来实现。

$$w'_{ijk} = s_i \cdot w_{ijk}, \quad (1)$$

其中， w 和 w' 分别表示原始权重和调制后的权重， s_i 表示与第 i 个输入特征图对应的缩放因子，而 j 和 k 则分别枚举卷积的输出特征图数量及其空间尺寸。

实例归一化的目的是基本上消除 s 对卷积输出特征图统计的影响。我们观察到，这一目标可以通过更直接的方式实现。假设输入激活是均值为零、标准差为1的独立同分布随机变量，在调制和卷积之后，输出激活的标准差为

$$\sigma_j = \sqrt{\sum_{i,k} w'_{ijk}{}^2}, \quad (2)$$

即输出被对应权重的 L_2 范数进行了缩放。后续的归一化目的是将输出恢复到单位标准差。根据公式2，如果我们对其进行缩放（“解调制”），即可实现这一目标。

每个输出特征图都乘以1。或者，我们也可以将这一点融入卷积权重中：

$$w''_{ijk} = w'_{ijk} / \sqrt{\sum_{i,k} w'_{ijk}{}^2 + \epsilon}, \quad (3)$$

其中 ϵ 是一个小常数，用于避免数值问题。

我们将整个风格块烘焙成一个单一的卷积层，其权重根据 s 通过公式1和3进行调整（图2d）。与实例归一化相比，我们的解调技术较弱，因为它基于信号统计假设而非特征图的实际内容。类似的统计分析在现代网络初始化中得到了广泛的应用[14, 19]，但我们不知道它之前被用作数据依赖性归一化的替代方法。此外，我们的解调与权重归一化[37]相关，后者作为重新参数化权重张量的一部分执行相同的计算。先前的工作已经确定，在生成对抗网络训练的背景下，权重归一化是有益的[43]。

我们的新设计消除了特征伪影（图3），同时保持了完全的可控性，如附带视频所示。FID基本上不受影响（表1，A、B行），但精度到召回率之间出现了显著转变。我们认为这种变化通常是有利的，因为可以通过截断操作将召回率转换为精度，而

Configuration	FFHQ, 1024×1024				LSUN Car, 512×384			
	FID ↓	Path length ↓	Precision ↑	Recall ↑	FID ↓	Path length ↓	Precision ↑	Recall ↑
A Baseline StyleGAN [24]	4.40	212.1	0.721	0.399	3.27	1484.5	0.701	0.435
B + Weight demodulation	4.39	175.4	0.702	0.425	3.04	862.4	0.685	0.488
C + Lazy regularization	4.38	158.0	0.719	0.427	2.83	981.6	0.688	0.493
D + Path length regularization	4.34	122.5	0.715	0.418	3.43	651.2	0.697	0.452
E + No growing, new G & D arch.	3.31	124.5	0.705	0.449	3.19	471.2	0.690	0.454
F + Large networks (StyleGAN2)	2.84	145.0	0.689	0.492	2.32	415.5	0.678	0.514
Config A with large networks	3.98	199.2	0.716	0.422	—	—	—	—

表1. 主要结果。对于每次训练运行，我们选择了FID值最低的训练快照。每个指标计算了10次，使用不同的随机种子，并报告其平均值。*Path length*对应于PPL指标，基于路径端点在 $\mathcal{W}$ [24]中计算得出，未采用Karras等人[24]使用的中心裁剪方法。FFHQ数据集包含70k张图像，在训练过程中判别器见到了25M张图像。对于LSUN CAR数据集，这两个数字分别为893k和57M。 $\uparrow$ 表示数值越高越好，而 $\downarrow$ 则表示数值越低越好。

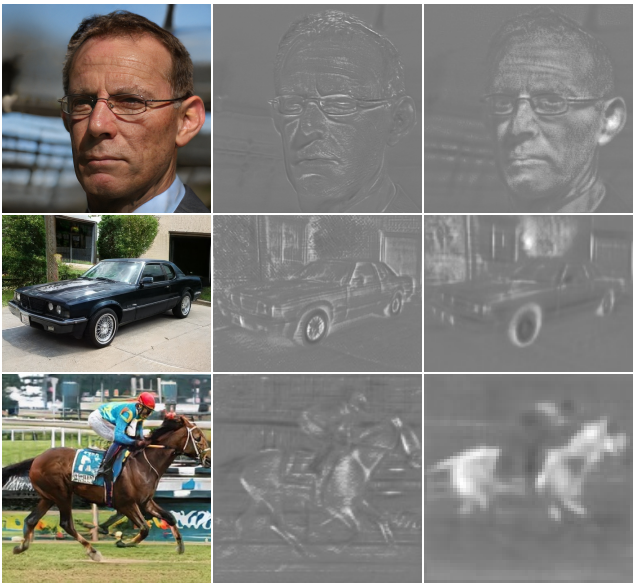


图3. 用调制替换归一化消除了图像和激活中的特征伪影。

相反，情况并非如此[27]。实际上，我们的设计可以高效地使用分组卷积来实现，详情参见附录B。为了在方程3中避免考虑激活函数的影响，我们将激活函数进行缩放，使其保持预期的信号方差。

3. 图像质量与生成器平滑度

虽然诸如FID或精度与召回率（P&R）之类的生成对抗网络（GAN）指标成功地捕捉到了生成器的许多方面，但它们仍然在一定程度上无法准确反映图像质量。例如，请参见图13和图14，这些图展示了具有相同FID和P&R得分但整体质量明显不同的生成器。

²We believe that the key to the apparent inconsistency lies in the particular choice of feature space rather than the foundations of FID or P&R. It was recently discovered that classifiers trained using ImageNet [35] tend to base their decisions much more on texture than shape [12], while humans strongly focus on shape [28]. This is relevant in our context because



图4. 使用基线StyleGAN（配置A）和LSUN CAT的数据集，感知路径长度与图像质量之间的关系。（a）PPL分数较低（第10百分位）的随机示例。（b）PPL分数较高（第90百分位）的示例。PPL评分与图像语义一致性之间存在明显的相关性。

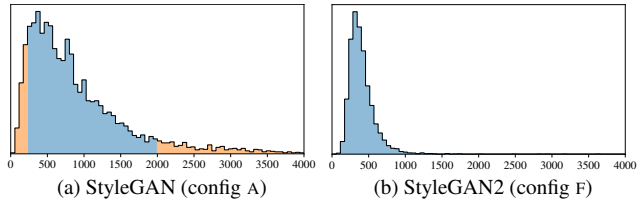


图5. （a）使用基线StyleGAN（配置A）生成的单个图像的PPL评分分布，该模型基于LSUN CAT数据集（FID = 8.53，PPL = 924）。图4中对应的百分位数范围用橙色标出。（b）StyleGAN2（配置F）显著改善了PPL评分分布（显示与相同FID = 8.53相对应的快照，PPL = 387）。

我们观察到感知图像质量与感知路径长度（PPL）[24]之间存在关联，这一指标最初是为了量化从潜在空间到输出图像的映射平滑度而引入的，通过测量在潜在空间中进行小扰动时生成图像之间的平均LPIPS距离来实现。再次参考图13和图14，较小的PPL（更平滑的生成器映射）似乎与整体更高的图像质量相关。

FID and P&R use high-level features from InceptionV3 [39] and VGG-16 [39], respectively, which were trained in this way and are thus expected to be biased towards texture detection. As such, images with, e.g., strong cat textures may appear more similar to each other than a human observer would agree, thus partially compromising density-based metrics (FID) and manifold coverage metrics (P&R).

该指标能够反映出变化，而其他度量标准则无法察觉这种变化。图4通过在LSUN CAT数据集上计算围绕 $\mathbf{w} \sim f(\mathbf{z})$ 的潜在空间样本的每张图像PPL得分，更详细地探讨了这一相关性。低分数确实表明高质量的图像，反之亦然。图5a显示了相应的直方图，并揭示了分布的长尾特征。模型的整体PPL值仅仅是这些每张图像PPL得分的期望值。我们始终计算整个图像的PPL值，而Karras等人[24]则使用较小的中心裁剪区域进行计算。

较低的PPL与图像质量之间的关联并不显而易见。我们假设，在训练过程中，当判别器惩罚受损图像时，生成器最直接的改进方式是有效地扩展能够产生高质量图像的潜在空间区域。这会导致低质量的图像被挤压到变化迅速的小范围潜在空间中。虽然这种方法在短期内可以提高平均输出的质量，但累积的扭曲会损害训练的动力学，并最终影响图像的质量。

显然，我们不能仅仅鼓励最小化的PPL，因为这会导致生成器趋向于一个召回率为零的退化解。相反，我们将描述一个新的正则项，旨在获得更平滑的生成器映射，而不具有上述缺点。由于由此产生的正则化项计算成本较高，我们首先描述一种适用于任何正则化技术的一般优化方法。

3.1. 惰性正则化

通常，主要损失函数（例如对数损失[16]）和正则化项（例如 R_1 [30]）被写成一个单一表达式，并且同时进行优化。我们观察到，正则化项可以比主要损失函数计算得更少，从而大大减少了它们的计算成本以及整体内存使用量。表1中的C行显示，在每16个小批量中仅执行一次 R_1 正则化不会造成任何损害，因此我们也采用相同策略来处理新的正则化项。附录B提供了实现细节。

3.2. 路径长度正则化

我们希望确保在 $\mathcal{W}$ 中固定大小的步长会导致图像中的非零、固定幅度的变化。可以通过在图像空间中随机方向上进行步进并观察相应的 $\mathbf{w}$ 梯度来实证测量这种理想情况与实际之间的偏差。无论是在 $\mathbf{w}$ 还是图像空间的方向上，这些梯度长度应基本相等，这表明从潜在空间到图像空间的映射是良态的[33]。

在单个 $\mathbf{w} \in \mathcal{W}$ 处，生成器映射 $g(\mathbf{w}) : \mathcal{W} \mapsto \mathcal{Y}$ 的局部度量缩放特性由雅可比矩阵 $\mathbf{J}_{\mathbf{w}} = \partial g(\mathbf{w}) / \partial \mathbf{w}$ 捕获。出于保持向量预期长度不变的愿望，

我们根据上述方向，将正则化项形式化为

$$\mathbb{E}_{\mathbf{w}, \mathbf{y} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} (\|\mathbf{J}_{\mathbf{w}}^T \mathbf{y}\|_2 - a)^2, \quad (4)$$

其中 $\mathbf{y}$ 是一些随机图像，像素强度服从正态分布，而 $\mathbf{w} \sim f(\mathbf{z})$ 中的 $\mathbf{z}$ 也服从正态分布。我们在附录C中证明，在高维情况下，当 $\mathbf{J}_{\mathbf{w}}$ 在全局尺度上正交时（即仅相差一个全局缩放因子），该先验概率达到最小值。一个正交矩阵保持长度不变，并且不会沿任何维度引入挤压效应。

为了避免显式计算雅可比矩阵，我们使用恒等式 $\mathbf{J}_{\mathbf{w}}^T \mathbf{y} = \nabla_{\mathbf{w}}(g(\mathbf{w}) \cdot \mathbf{y})$ ，该恒等式可以通过标准反向传播[6]高效计算。常数 a 在优化过程中动态设置为长度 $\|\mathbf{J}_{\mathbf{w}}^T \mathbf{y}\|_2$ 的长期指数移动平均值，从而使优化过程能够自行找到合适的全局尺度。

我们的正则化器与Odena等人提出的雅可比矩阵钳制正则化器[33]密切相关。实际差异在于我们通过解析方法计算 $\mathbf{J}_{\mathbf{w}}^T \mathbf{y}$ ，而他们使用有限差分法来估计 $\mathbf{J}_{\mathbf{w}} \delta$ 中的 $\mathbf{z} \ni \delta \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 。需要注意的是，生成器的谱归一化[31]仅约束最大的奇异值，并不对其他奇异值施加限制，因此不一定能改善条件数。我们发现，在我们的贡献基础上启用谱归一化——或者用它替代我们的贡献——不可避免地会损害FID性能，详情见附录E。

在实践中，我们注意到路径长度正则化能够使模型更加可靠且行为一致，从而使得架构探索变得更加容易。我们还观察到，平滑的生成器更容易被逆向求解（见第5节）。图5b显示，路径长度正则化明显收紧了每张图像PPL评分的分布，并未将模式推向零。然而，表1中的D行表明，在结构不如FFHQ的数据集上，FID和PPL之间存在权衡。

4. 进度增长再探

渐进增长法[23]在稳定高分辨率图像合成方面非常成功，但也会产生其特有的伪影。关键问题在于逐步生长的生成器似乎对细节的位置有强烈的偏好；伴随视频显示，在特征如牙齿或眼睛应该平滑地移动时，它们可能会卡在一个位置上，然后突然跳到下一个首选位置。图6展示了相关的伪影。我们认为问题在于渐进增长法中每个分辨率暂时作为输出分辨率使用，迫使生成器产生最高频的细节，这导致训练后的网络在中间层具有过高的频率，从而破坏了平移不变性[49]。附录A给出了一个例子。这些



图6. 进度增长导致“阶段”伪影。如示例所示，牙齿不随姿态变化，而是与相机保持对齐，如蓝色线条所示。

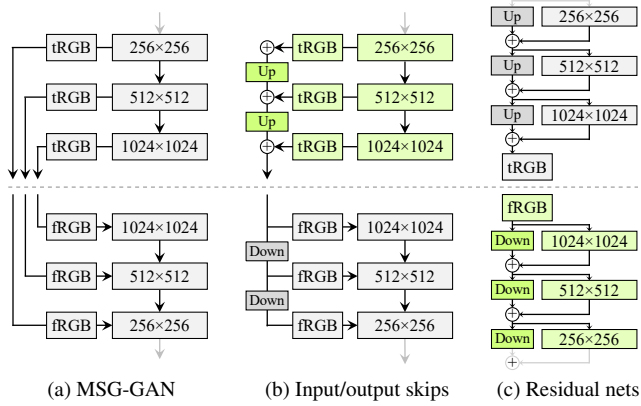


图7. 三个生成器（虚线以上部分）和判别器架构。Up 和 Down 分别表示双线性上采样和下采样。在残差网络中，这些还包括 $1 \times 1 \times 1$ 卷积以调整特征图的数量。tRGB 和 fRGB 负责在 RGB 数据与高维像素数据之间进行转换。配置 E 和 F 中使用的架构用绿色标出。

问题促使我们寻求一种替代方案，在保留渐进增长带来的好处的同时避免其缺点。

4.1. 替代网络架构

虽然StyleGAN在生成器（合成网络）和判别器中使用简单的前馈设计，但关于更好网络架构的研究却十分丰富。跳跃连接[34, 22]、残差网络[18, 17, 31]以及分层方法[7, 47, 48]在生成方法的背景下也取得了显著的成功。因此，我们决定重新评估StyleGAN的网络设计，并寻找一种无需逐步增长即可产生高质量图像的架构。

图7a展示了MSG-GAN[22]，该模型通过多个跳跃连接来匹配生成器和判别器的分辨率。将MSG-GAN生成器修改为输出一个 mipmap[42] 而不是图像，并且对于每个真实图像计算类似的表示。

FFHQ	D original		D input skips		D residual	
	FID	PPL	FID	PPL	FID	PPL
G original	4.32	265	4.18	235	3.58	269
G output skips	4.33	169	3.77	127	3.31	125
G residual	4.35	203	3.96	229	3.79	243

LSUN Car	D original		D input skips		D residual	
	FID	PPL	FID	PPL	FID	PPL
G original	3.75	905	3.23	758	3.25	802
G output skips	3.77	544	3.86	316	3.19	471
G residual	3.93	981	3.40	667	2.66	645

表2. 不采用渐进增长的生成器和判别器架构对比。带有输出跳跃连接的生成器与残差判别器的组合对应于主结果表中的配置E。

年龄也是如此。如图7b所示，我们通过上采样并求和不同分辨率对应的RGB输出的贡献来简化此设计。在判别器中，我们将下采样的图像提供给判别器中的每个分辨率块。我们在所有上采样和下采样操作中使用双线性滤波。如图7c所示，我们进一步修改了设计以使用残差连接。这种设计与LAPGAN [7] 类似，但没有采用Denton等人使用的每种分辨率的判别器。

表2比较了三种生成器和三种判别器架构：原始的前馈网络（如StyleGAN中使用的）、跳跃连接以及残差网络，所有这些均未采用渐进式增长训练。对于每种组合，提供了FID和PPL指标。我们可以观察到两个主要趋势：在生成器中使用跳跃连接可以显著提高所有配置下的PPL；而残差判别器网络显然有利于提升FID值。后者或许并不令人惊讶，因为判别器的结构类似于分类器，在这种情况下，已知残差架构是有益的。然而，在生成器中采用残差架构却有害——唯一的例外是在LSUN CAR数据集上，当两个网络均为残差时，FID指标有所改善。

本文余下部分采用跳过生成器和残差判别器，不进行渐进式增长。这对应于表1中的配置E，并且显著提升了FID和PPL。

4.2. 分辨率使用

渐进增长的关键在于生成器最初会专注于低分辨率特征，然后逐渐将注意力转移到更精细的细节上。如图7所示的架构使得生成器可以首先输出不受高分辨率层显著影响的低分辨率图像，随后再逐步转向高分辨率层面。

³In residual network architectures, the addition of two paths leads to a doubling of signal variance, which we cancel by multiplying with $1/\sqrt{2}$. This is crucial for our networks, whereas in classification resnets [18] the issue is typically hidden by batch normalization.

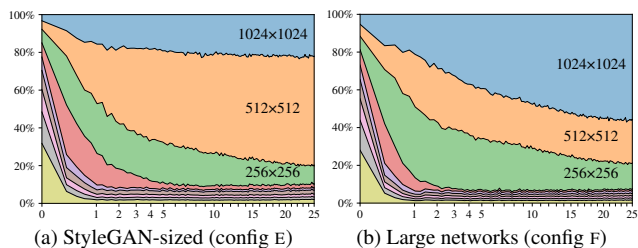


图8. 每种分辨率对生成器输出的贡献随训练时间的变化。纵轴显示了不同分辨率的标准偏差分解，横轴对应于以展示给判别器的训练图像数量（百万计）衡量的训练进度。可以看到，在开始时网络主要关注低分辨率图像，并随着训练进展逐渐转向更高分辨率的关注。在(a)中，生成器基本上输出一个 512^2 图像，并对 1024^2 进行一些轻微锐化处理；而在(b)中，更大的网络则更加注重高分辨率细节。

随着训练的进行，生成器会逐渐依赖不同层次的分辨率。由于没有强制要求，生成器只会根据有利情况来这样做。为了分析其实际行为，我们需要量化生成器在整个训练过程中对特定分辨率的依赖程度。

由于跳过生成器（图7b）通过显式地从多个分辨率中求和RGB值来形成图像，我们可以通过测量它们对最终图像的贡献程度来估计相应层的重要性。在图8a中，我们绘制了每个tRGB层产生的像素值的标准差作为训练时间的函数。我们计算了1024个随机样本 w 的标准差，并将这些值归一化，使其总和为100%。

在训练初期，我们可以观察到新的跳跃生成器的行为类似于逐步增长——现在无需改变网络拓扑结构即可实现。因此可以合理地预期，在训练的最后阶段最高分辨率将占据主导地位。然而，图表显示这种情况实际上并未发生，这表明生成器可能无法“充分利用”目标分辨率。为了验证这一点，我们手动检查了生成的图像，并注意到它们通常缺少一些在训练数据中存在的像素级细节——这些图像更像是 512^2 图像的锐化版本，而不是真正的 1024^2 图像。

这使我们假设我们的网络存在容量问题，通过将两个网络中最高分辨率层的特征图数量翻倍来验证这一假设。这样可以使行为更符合预期。

⁴We double the number of feature maps in resolutions 64^2 – 1024^2 while keeping other parts of the networks unchanged. This increases the total number of trainable parameters in the generator by 22% ($25M \rightarrow 30M$) and in the discriminator by 21% ($24M \rightarrow 29M$).

Dataset	Resolution	StyleGAN (A)		StyleGAN2 (F)	
		FID	PPL	FID	PPL
LSUN CAR	512×384	3.27	1485	2.32	416
LSUN CAT	256×256	8.53	924	6.93	439
LSUN CHURCH	256×256	4.21	742	3.86	342
LSUN HORSE	256×256	3.83	1405	3.43	338

表3. 使用FID和PPL在LSUN数据集上测量的改进。我们对CAR训练了57 million张图像，CAT训练了88 million张，CHURCH训练了48 million张，HORSE训练了100 million张图像。

图8b显示了最高分辨率层的贡献显著增加，表1中的第F行表明FID和Recall指标有了明显改善。最后一行显示，基础版StyleGAN也从额外的容量中受益，但其质量仍然远低于StyleGAN2。

表3比较了StyleGAN和StyleGAN2在四个LSUN类别的表现，再次展示了FID的明显改进以及PPL的显著进步。进一步增加模型规模可能带来额外的好处。

5. 图像到潜在空间的投影

反转合成网络是一个有趣的问题，具有许多应用。在潜在特征空间中操作给定图像需要首先找到与其匹配的潜在代码。先前的研究[1, 10]表明，与寻找一个通用的潜在代码相比，为生成器的每一层选择单独的潜在代码可以得到更好的结果。早期编码器实现[32]也采用了相同的方法。虽然以这种方式扩展潜在空间能够更接近地匹配给定图像，但它还允许将任意没有潜在表示的图像进行投影。相反，我们专注于在原始未扩展的潜在空间中寻找潜在代码，因为这些潜在代码对应于生成器可能产生的图像。

我们的投影方法与以往的方法有两点不同。首先，在优化过程中，我们在潜在代码中加入逐渐减弱的噪声，以更全面地探索潜在空间。其次，我们还优化了StyleGAN生成器中的随机噪声输入，并对其进行正则化处理，确保这些噪声不会携带连贯信号。该正则化基于强制噪声图的自相关系数在多个尺度上与单位高斯噪声相匹配。我们的投影方法的具体细节可参见附录D。

5.1. 生成图像的归属认定

检测操纵或生成的图像是一项非常重要的任务。目前，基于分类器的方法可以相当可靠地检测出生成的图像，无论其确切来源如何[29, 45, 40, 51, 41]。然而，鉴于生成方法的进步速度很快，这种情况可能不会持久。除了对假图像进行一般的检测外，我们还可以考虑该问题的一个更为有限的形式：



图9. 示例图像及其投影和重新合成的对应图像。对于每种配置，第一行显示目标图像，第二行显示相应投影潜在向量和噪声输入的合成结果。使用基线StyleGAN时，投影通常能找到生成图像的一个合理匹配，但尤其是背景与原图有所不同。而使用StyleGAN2生成的图像几乎可以完美地重新投影回生成器输入中，而实际训练集中的真实图像（经过投影）则明显不同于原始图像，符合预期。所有测试均采用相同的投影方法和超参数进行。

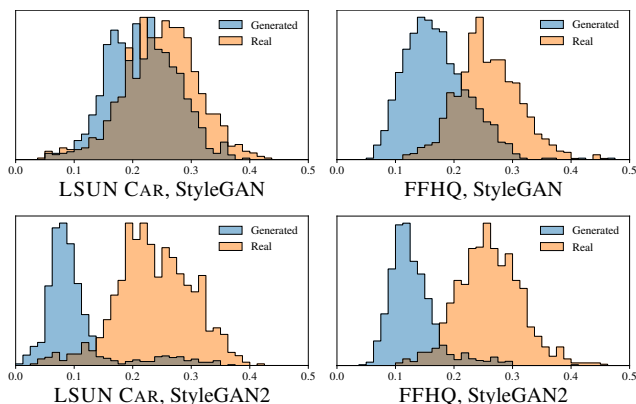


图10. 原始图像与生成图像之间的LPIPS距离直方图（生成图像为蓝色，真实图像为橙色）。尽管我们的改进生成器提高了图像质量，但将生成的图像投影到其潜在空间 $\mathcal{W}$ 中要容易得多。在所有情况下均使用了相同的投影方法。

能够将一张伪造图像归因于其特定来源[2]。使用StyleGAN，这相当于检查是否存在一个 $\mathbf{w} \in \mathcal{W}$ 来重新合成该问题图像。

我们通过计算原始图像与重新合成图像之间的LPIPS [50]距离来衡量投影的成功程度，记为 $D_{\text{LPIPS}}[\mathbf{x}, g(\tilde{g}^{-1}(\mathbf{x}))]$ ，其中 $\mathbf{x}$ 是待分析的图像，而 $\tilde{g}^{-1}$ 表示近似投影操作。图10展示了使用原始StyleGAN和StyleGAN2时LSUN CAR及FFHQ数据集这些距离的直方图分布情况，图9则显示了示例投影结果。使用StyleGAN2生成的图像可以如此精确地投影到 $\mathcal{W}$ 中，以至于几乎可以无歧义地归因于生成网络。然而，对于原始StyleGAN而言，尽管从技术上讲应该能够找到匹配的潜在代码，但似乎从 $\mathcal{W}$ 到图像的映射过于复杂，无法在实践中可靠地实现这一目标。我们发现即使图像质量显著提高，StyleGAN2仍使源归属变得更加容易，这一点令人鼓舞。

6. 结论与未来工作

我们已在StyleGAN中识别并修复了若干图像质量问题，进一步提升了图像质量，并在多个数据集上显著推进了当前技术水平。在某些情况下，改进效果在动态演示中更为明显，相关视频可作为参考。附录A提供了使用本方法所能获得的更多结果示例。尽管质量有所提升，StyleGAN2使得将生成的图像归因于其来源变得更加容易。

训练性能也得到了提升。在1024²分辨率下，原始的StyleGAN（表1中的配置A）在配备8块Tesla V100 GPU的NVIDIA DGX-1上以每秒37张图像的速度进行训练，而我们的配置E则快40%，达到每秒61张图像。大部分加速来自于权重解调、惰性正则化以及代码优化所简化后的数据流。StyleGAN2（配置F，更大的网络）的训练速度为每秒31张图像，因此其训练成本仅略高于原始StyleGAN。在FFHQ和LSUN CAR数据集上的总训练时间分别为9天和13天。

该项目包括所有探索阶段，共消耗了132兆瓦时的电力，其中0.68兆瓦时用于训练最终的FFHQ模型。总计我们使用了大约51年的单GPU计算时间（伏尔塔级GPU）。更多详细讨论见附录F。

未来的研究可以进一步探索改进路径长度正则化的方法，例如通过用数据驱动的特征空间度量替换像素空间的距离。考虑到生成对抗网络（GAN）的实际部署，我们认为找到新的方法来减少训练数据的需求将非常重要。特别是在那些难以获取数万份训练样本的应用中，以及在包含大量内在变化的数据集中，这一点尤为重要。

致谢 我们感谢 Ming-Yu Liu 的早期审阅，感谢 Timo Vitanen 在公共发布方面的帮助，感谢 David Luebke 深入的讨论和有益的意见，以及感谢 Tero Kuosmanen 在计算基础设施方面提供的技术支持。

参考文献

[1] Rameen Abdal, Yipeng Qin 和 Peter Wonka. Image2Style GAN: 如何将图像嵌入到 StyleGAN 的潜在空间中? 在 *ICCV* 中, 2019 年. 7[2] Michael Albright 和 Scott McCloskey. 通过逆向工程进行源生成器归因. 在 *CVPR Workshops* 中, 2019 年. 8[3] Carl Bergstrom 和 Jevin West. 哪一张脸是真实的? <http://www.whichfaceisreal.com/learn.html>, 访问日期: 2019 年 11 月 15 日. 1[4] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer 出版社, 2006 年. 17[5] Andrew Brock, Jeff Donahue 和 Karen Simonyan. 大规模 GAN 训练以实现高保真度的自然图像合成. *CoRR*, arXiv:1809.11096, 2018 年. 1[6] Yann N. Dauphin, Harm de Vries 和 Yoshua Bengio. 非凸优化中的平衡自适应学习率. *CoRR*, arXiv:1502.04390, 2015 年. 5[7] Emily L. Denton, Soumith Chintala, Arthur Szlam 和 Robert Fergus. 使用拉普拉斯金字塔对抗网络的深度生成图像模型. *CoRR*, arXiv:1506.05751, 2015 年. 6[8] Vincent Dumoulin, Ethan Perez, Nathan Schucher 和 Florian Strub. Harm de Vries, Aaron Courville 和 Yoshua Bengio. 特征级变换. *Distill*, 2018 年. <https://distill.pub/2018/feature-wise-transformations>. 1[9] Vincent Dumoulin, Jonathon Shlens 和 Manjunath Kudlur. 学习的艺术风格表示形式. *CoRR*, arXiv:1610.07629, 2016 年. 1[10] Aviv Gabbay 和 Yedid Hoshen. 用于图像增强和动画的样式生成器逆向工程. *CoRR*, arXiv:1906.11880, 2019 年. 7[11] R. Ge, X. Feng, H. Pyla, K. Cameron 和 W. Feng. Green500 列表中的功率测量教程. <https://www.top500.org/green500/resources/tutorials/>, 访问日期: 2020 年 3 月 1 日. 21[12] Robert Geirhos, Patricia Rubisch, Claudio Michaelis, Matthias Bethge, Felix A. Wichmann 和 Wieland Brendel. ImageNet 训练的 CNN 倾向于纹理; 增加形状偏置可提高准确性和鲁棒性. *CoRR*, arXiv:1811.12231, 2018 年. 1, 4[13] Golnaz Ghiasi, Honglak Lee, Manjunath Kudlur, Vincent Dumoulin 和 Jonathon Shlens. 探索实时任意神经艺术风格化网络的结构. *CoRR*, arXiv:1705.06830, 2017 年. 1[14] Xavier Glorot 和 Yoshua Bengio. 理解深层前馈神经网络训练的难度. 在 *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics* 中, 第 249-256 页, 2010 年. 3[15] G.H. Golub 和 C.F. Van Loan. *Matrix Computations*. 约翰霍普金斯数学科学研究丛书. 约翰霍普金斯大学出版社, 2013 年. 17[16] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville 和 Yoshua Bengio. 生成对抗网络. 在 *NIPS* 中, 2014 年. 1, 5, 11

[17] Ishaan Gulrajani, Faruk Ahmed, Martín Arjovsky, Vincent Dumoulin 和 Aaron C. Courville. 改进的 Wasserstein GAN 训练方法. arXiv:1704.00028, 2017. 6[18] Kaiming He, Xiangyu Zhang, Shaoqing Ren 和 Jian Sun. 深度残差学习在图像识别中的应用. arXiv:1512.03385, 2015. 6[19] Kaiming He, Xiangyu Zhang, Shaoqing Ren 和 Jian Sun. 探索整流器: 超越人类水平的 ImageNet 分类性能. arXiv:1502.01852, 2015. 3[20] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler 和 Sepp Hochreiter. GANs 通过双时间尺度更新规则训练收敛于局部纳什均衡. 在某会议论文集, 第 6626-6637 页, 2017 年. 1[21] Xun Huang 和 Serge J. Belongie. 实时任意风格迁移: 自适应实例归一化方法. arXiv:1703.06868, 2017. 1[22] Animesh Karnewar 和 Oliver Wang. MSG-GAN: 用于生成对抗网络的多尺度梯度技术. 在某会议论文集, 2020 年. 6[23] Tero Karras, Timo Aila, Samuli Laine 和 Jaakko Lehtinen. GANs 渐进式增长以提高质量、稳定性和多样性. arXiv:1710.10196, 2017 年. 1, 5, 11[24] Tero Karras, Samuli Laine 和 Timo Aila. 基于风格的生成对抗网络架构. 在某会议论文集, 2018 年. 1, 2, 4, 5, 11, 13, 16, 20[25] Diederik P. Kingma 和 Jimmy Ba. Adam: 一种随机优化方法. 在某会议论文集, 2015 年. 11, 19[26] Alex Krizhevsky, Ilya Sutskever 和 Geoffrey E. Hinton. 使用深度卷积神经网络进行 ImageNet 分类. 在某会议论文集, 第 1097-1105 页, 2012 年. 11[27] Tuomas Kynkäänniemi, Tero Karras, Samuli Laine, Jaakko Lehtinen 和 Timo Aila. 改进的生成模型评估方法: 精度和召回率指标. 在某会议论文集, 2019 年. 1, 2, 4[28] Barbara Landau, Linda B. Smith 和 Susan S. Jones. 形状在早期词汇学习中的重要性. 某期刊, 3(3), 1988 年. 4[29] Haodong Li, Han Chen, Bin Li 和 Shunquan Tan. 鉴别器能否识别 GAN 生成的图像? 在某会议论文集, 2018 年. 7[30] Lars Mescheder, Andreas Geiger 和 Sebastian Nowozin. 哪些训练方法能使 GAN 实际收敛? arXiv:1801.04406, 2018 年. 5, 11[31] Takeru Miyato, Toshiki Kataoka, Masanori Koyama 和 Yuichi Yoshida. 对生成对抗网络的谱归一化技术. arXiv:1802.05957, 2018 年. 1, 5, 6, 20[32] Dmitry Nikitko. StyleGAN – TensorFlow 官方实现编码器. <https://github.com/Puzer/stylegan-encoder/>, 2019 年. 7[33] Augustus Odena, Jacob Buckman, Catherine Olsson, Tom B. Brown, Christopher Olah, Colin Raffel 和 Ian Goodfellow. 生成器条件化是否与 GAN 性能因果相关? arXiv:1802.08768, 2018 年. 5, 18

[34] Olaf Ronneberger, Philipp Fischer 和 Thomas Brox。U-Net: 用于生物医学图像分割的卷积网络。在 *Proc. Medical Image Computing and Computer-Assisted Intervention (MICCAI)* 中, 第 234–241 页, 2015 年。

6[35] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael S. Bernstein, Alexander C. Berg 和 Fei-Fei Li。ImageNet 大规模视觉识别挑战。在 *Proc. CVPR* 中, 2015 年。

4[36] Mehdi S. M. Sajjadi, Olivier Bachem, Mario Lucic, Olivier Bousquet 和 Sylvain Gelly。通过精度和召回率评估生成模型。 *CoRR*, abs/1806.00035, 2018 年。

1[37] Tim Salimans 和 Diederik P. Kingma。权重归一化: 一种简单的重新参数化方法, 用于加速深度神经网络的训练。 *CoRR*, abs/1602.07868, 2016 年。

3[38] Yujun Shen, Jinjin Gu, Xiaoou Tang 和 Bolei Zhou。解释 GAN 的潜在空间以进行语义面部编辑。 *CoRR*, abs/1907.10786, 2019 年。

1[39] Karen Simonyan 和 Andrew Zisserman。用于大规模图像识别的非常深卷积网络。 *CoRR*, abs/1409.1556, 2014 年。

1, 4[40] Run Wang, Lei Ma, Felix Juefei-Xu, Xiaofei Xie, Jian Wang 和 Yang Liu。FakeSpotter: 一种简单的基线方法用于检测 AI 合成的假脸。 *CoRR*, abs/1909.06122, 2019 年。

7[41] Sheng-Yu Wang, Oliver Wang, Richard Zhang, Andrew Owens 和 Alexei A. Efros。CNN 生成的图像令人惊讶地容易被识别……至少目前是这样。 *CoRR*, abs/1912.11035, 2019 年。

7[42] Lance Williams。金字塔参数化。 *SIGGRAPH Comput. Graph.*, 第 17 卷第 3 期: 1–11, 1983 年。

6[43] Sitao Xiang 和 Hao Li。批量归一化和权重归一化在生成对抗网络中的影响研究。 *CoRR*, abs/1704.03971, 2017 年。

3[44] Fisher Yu, Yinda Zhang, Shuran Song, Ari Seff 和 Jianxiong Xiao。LSUN: 使用深度学习和人类参与构建大规模图像数据集。 *CoRR*, abs/1506.03365, 2015 年。

11[45] Ning Yu, Larry Davis 和 Mario Fritz。将假图像归因于 GANs: 分析生成图像中的指纹。 *CoRR*, abs/1811.08180, 2018 年。

7[46] Han Zhang, Ian Goodfellow, Dimitris Metaxas 和 Augustus Odena。自注意力生成对抗网络。 *CoRR*, abs/1805.08318, 2018 年。

5[47] Han Zhang, Tao Xu, Hongsheng Li, Shaoting Zhang, Xiaolei Huang, Xiaogang Wang 和 Dimitris N. Metaxas。StackGAN: 使用堆叠生成对抗网络将文本转换为逼真图像的合成方法。在 *ICCV* 中, 2017 年。

6[48] Han Zhang, Tao Xu, Hongsheng Li, Shaoting Zhang, Xiaogang Wang, Xiaolei Huang 和 Dimitris N. Metaxas。StackGAN: 使用堆叠生成对抗网络实现逼真图像合成的方法。 *CoRR*, abs/1710.10916, 2017 年。

6[49] Richard Zhang。再次使卷积网络具有平移不变性。在 *Proc. ICML* 中, 2019 年。

5, 11[50] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman 和 Oliver Wang。深度特征作为感知度量的不合理有效性。在 *Proc. CVPR* 中, 2018 年。

4, 8, 19

张旭, 卡拉曼斯韦博尔, 和张世富。检测和模拟GAN生成图像中的伪影。 *arXiv:abs/1907.06515*, 2019. 7

A. 图像质量

我们包含了几幅较大的图片，用以说明与图像质量相关的各个方面。图11展示了精心挑选的示例，这些示例体现了在FFHQ数据集上使用我们的方法所能达到的质量和多样性，而图12则显示了论文中提到的所有数据集未经筛选的结果。

图13和图14展示了FID和P&R给出非直观结果的情况，但PPL似乎更符合人类的判断。

我们也包括了与StyleGAN生成瑕疵相关的图片。图15展示了一个罕见的案例，在此情况下，团块瑕疵未能出现在StyleGAN激活中，导致图像严重损坏。图16展示了表1配置A和F中的激活情况。很明显，渐进式增长会导致中间层高频内容增加，从而破坏网络的平移不变性。我们假设这造成了在使用渐进式增长时观察到的细节位置偏好不均匀现象。

B. 实现细节

我们在 TensorFlow 官方 StyleGAN⁵ 的配置 A 对应的实现基础上实现了我们的技术。我们保留了大部分细节不变，包括 $\mathcal{Z}$ 和 $\mathcal{W}(512)$ 的维度、映射网络架构（8 个全连接层， $100 \times$ 学习率较低）、所有可训练参数的均衡学习率 [23]、具有 $\alpha = 0.2$ 泄漏 ReLU 激活函数、在所有上采样和下采样层中使用双线性滤波器 [49] [24]，判别器末尾处的小批量标准差层 [23]，生成器权重的指数移动平均值 [23]，风格混合正则化 [24]，非饱和逻辑损失 [16] 以及 R_1 正则化 [30]，使用与 [25] 相同超参数（ $\beta_1 = 0, \beta_2 = 0.99, \epsilon = 10^{-8}$ ，小批量大小为 32）的 Adam 优化器，并且使用了训练数据集 [24, 44]。所有训练运行均在配备 8 块 Tesla V100 GPU 的 NVIDIA DGX-1 上进行，使用 TensorFlow 1.14.0 和 cuDNN 7.4.2。

生成器重新设计 在配置 B–F 中，我们将原始的 StyleGAN 生成器替换为我们的修订架构。除了第二节中提到的变化外，我们还使用 $\mathcal{N}(0, 1)$ 初始化常量输入 c_1 的组件，并简化噪声广播操作以使用所有特征图共用的一个共享缩放因子。类似于 Karas 等人 [24] 的做法，我们将所有权重初始化为 $\mathcal{N}(0, 1)$ ，并将所有偏置和噪声缩放因子初始化为零（除了仿射变换层的偏置项，我们将其初始化为 1）。我们在所有卷积层中使用权重调制和解调，输出层 (tRGB) 除外。

图7所示，此处省略了解调过程。输出分辨率为1024时，生成器总共包含18个仿射变换层，其中第一层对应4分辨率，接下来的两层对应8分辨率，以此类推。

权重解调 考虑到实现的实际实现，需要注意的是，对于小批量中的每个样本，得到的权重集会有所不同，这排除了直接使用标准卷积原语进行实现的可能性。相反，我们选择采用 *grouped convolutions* [26] 的方法，该方法最初提出通过将输入特征图划分为多个独立组来降低计算成本，每组都有自己的专用权重集。我们通过对权重和激活暂时重新整形来实现方程1和3，使得每次卷积只看到一个样本的 N 个组——而不是 N 个样本的一个组。这种方法非常高效，因为重新整形操作实际上不会修改权重和激活张量的内容。

懒惰正则化 在配置中，我们采用懒惰正则化（参见第3.1节），通过在一个单独的正则化遍历中评估正则化项（ R_1 和路径长度）来实现。我们在每 k 次训练迭代后执行一次这样的正则化遍历。主损失函数与正则化项共享Adam优化器的内部状态，因此优化器首先在前 k 次迭代中看到来自主损失函数的梯度，然后在接下来的一次迭代中看到来自正则化项的梯度。为了补偿我们现在执行的是每 $k+1$ 次训练迭代而不是 k 的情况，我们调整了优化器超参数 $\lambda' = c \cdot \lambda$ 、 $\beta'_1 = (\beta_1)^c$ 和 $\beta'_2 = (\beta_2)^c$ ，其中 $c = k/(k+1)$ 。同时，我们将正则化项乘以 k 来平衡其梯度的整体幅度。对于判别器我们使用 $k = 16$ ，生成器则使用 $k = 8$ 。

路径长度正则化 配置 D–F 包含了我们新的路径长度正则器（参见第 3.2 节）。我们将目标尺度 a 初始化为零，并以每块 GPU 的方式跟踪其指数移动平均值，该平均值基于 $\|\mathbf{J}_{\mathbf{w}}^T \mathbf{y}\|_2$ ，衰减系数为 $\beta_{\text{pl}} = 0.99$ 。我们通过权重来调整正则化项的大小。

$$\gamma_{\text{pl}} = \frac{\ln 2}{r^2(\ln r - \ln 2)}, \quad (5)$$

其中 r 指定输出分辨率（例如 $r = 1024$ ）。我们发现这些参数选择在所有配置和数据集上都能可靠地工作。为了确保我们的正则化项能正确与风格混合正则化相互作用，我们将它计算为合成网络中所有单独层的平均值。附录 C 提供了关于我们正则化项对从 $\mathcal{W}$ 到图像空间映射影响的详细分析。

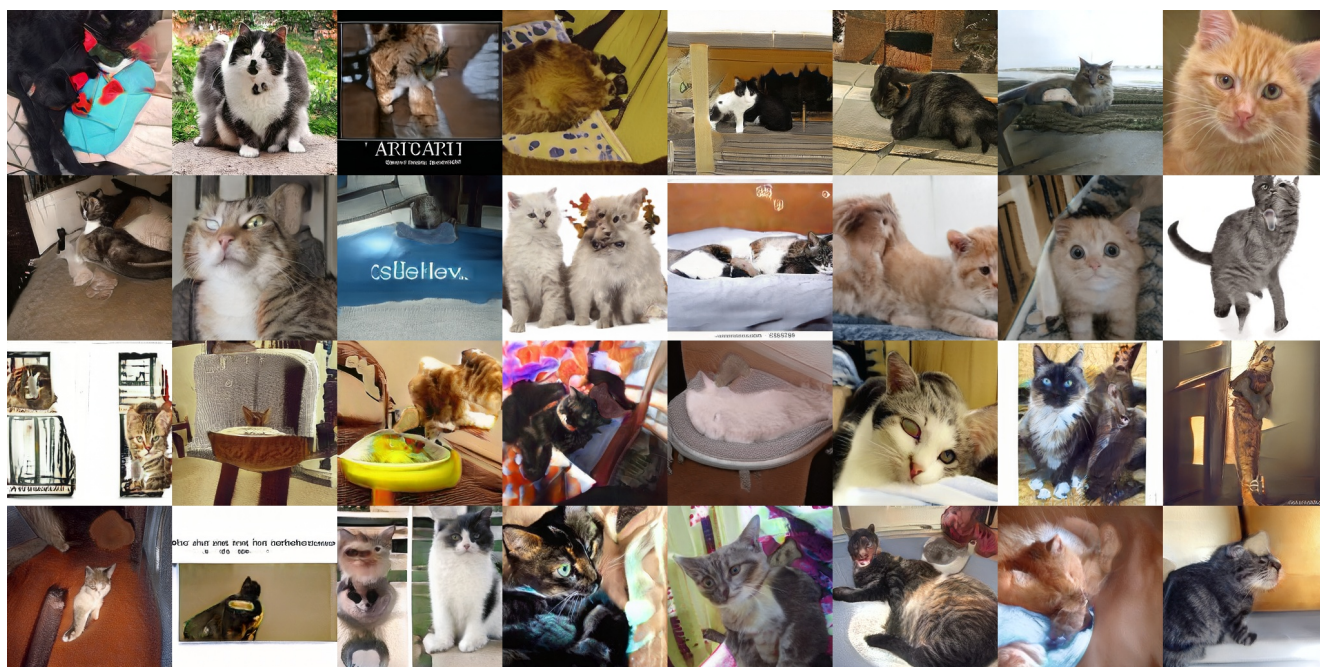
⁵<https://github.com/NVlabs/stylegan>



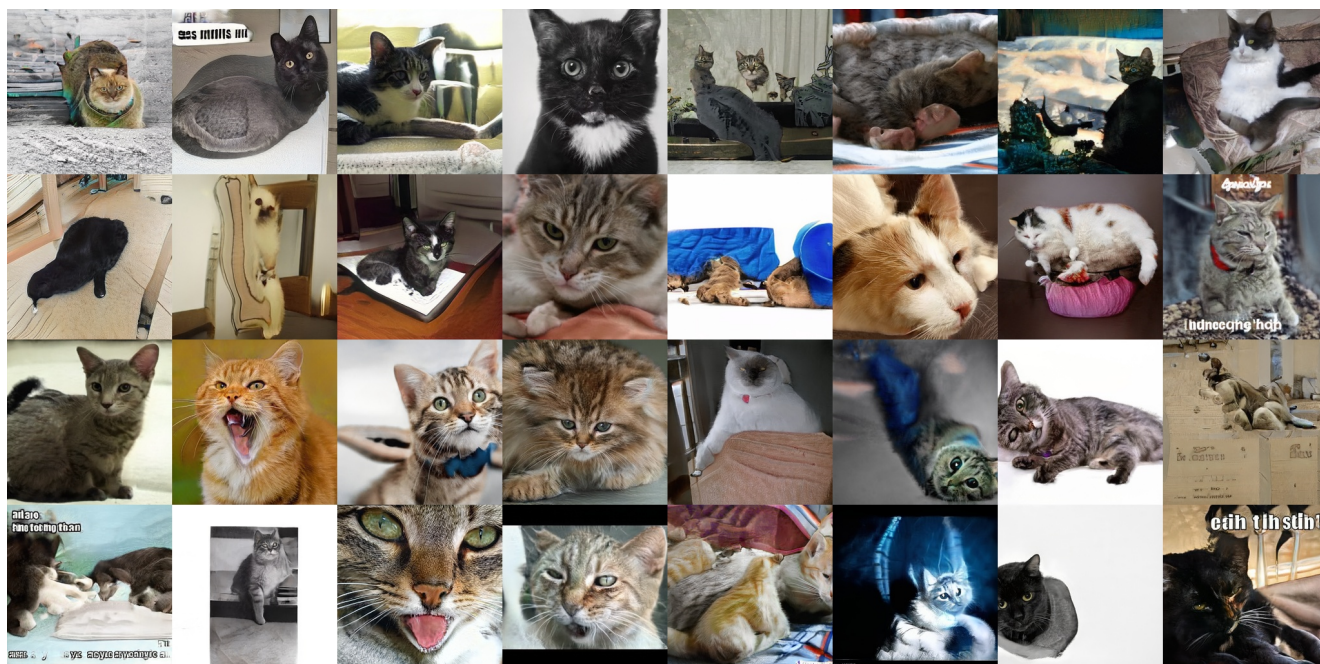
图11. 四个精选示例，展示了使用StyleGAN2（配置F）所能实现的图像质量和多样性。



图12. 表1和表3中使用的每个数据集的未整理结果。图像对应于我们的生成器（配置F）在所有分辨率下使用 $\psi = 0.5$ [24]进行截断处理后产生的随机输出。



Model 1: FID = 8.53, P = 0.64, R = 0.28, PPL = 924



模型2: FID = 8.53, P = 0.62, R = 0.29, PER = 387

图13. 在LSUN CAT数据集上训练且未进行截断的两个生成模型的原始样本。尽管后者更频繁地产生猫形物体，模型1和模型2的FID、精度和召回率相似。感知路径长度（PPL）表明偏好于模型2。模型1对应表3中的配置A，而模型2是配置F早期训练快照的结果。

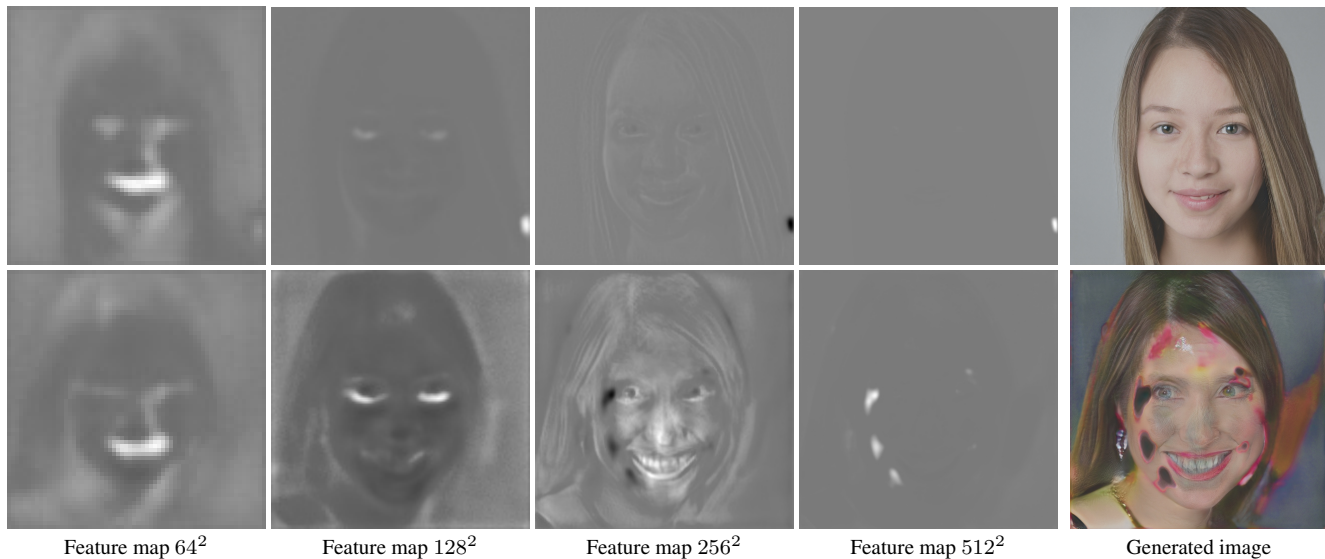


图15 展示了StyleGAN生成器中液滴伪影的重要性。我们比较了两张生成的图像，一张成功生成而另一张严重受损。对应的特征图通过实例归一化处理显示在可观察动态范围内。对于上图，在 64^2 分辨率下开始出现液滴伪影，在 128^2 分辨率下清晰可见，并且随着分辨率提高逐渐主导整个特征图。而对于下图， 64^2 的图像与上方行类似，但在 128^2 分辨率下没有形成液滴。因此，面部特征在归一化后的特征图中更加明显。这导致在 256^2 分辨率时出现过冲现象，并且随后的多个分辨率中出现了多个虚假液滴。根据我们的经验，在StyleGAN生成的图像中缺少液滴伪影的情况非常罕见，事实上生成器完全依赖于其存在。

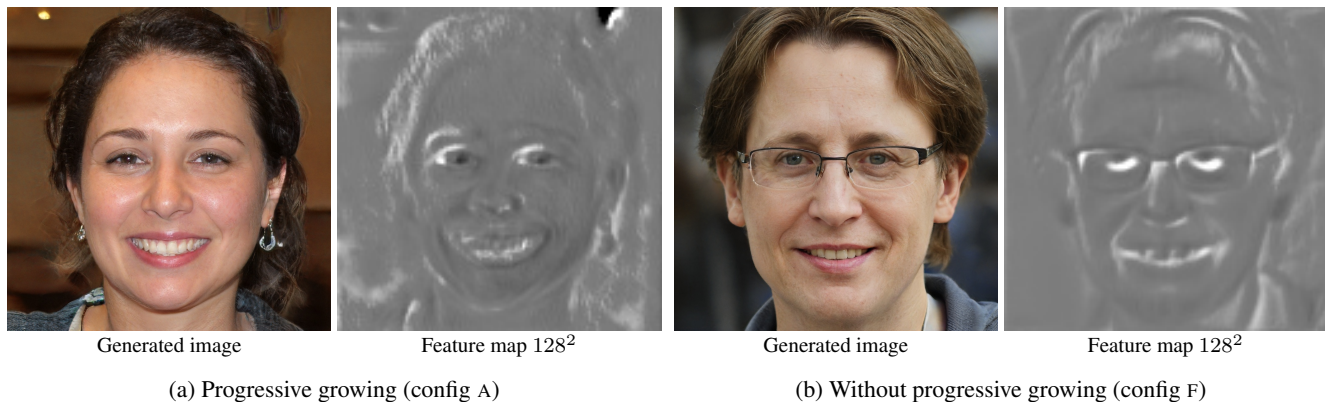


图16. 进度增长导致中间层高频内容显著增加。这会损害网络的平移不变性，并使得在高分辨率层中精确定位特征变得更加困难。

在配置A-D中，我们使用与Karasz等人[24]相同的渐进增长参数（从 8^2 分辨率和学习率 $\lambda = 10^{-3}$ 开始，每种分辨率训练60万张图像，下一分辨率的淡入时间为60万张图像，并逐渐将学习率提高 $3\times$ ）。在配置E-F中，我们禁用渐进增长并将学习率设置为固定值 $\lambda = 2 \cdot 10^{-3}$ ，这是我们发现的最佳结果。此外，我们在生成器中使用输出跳跃连接，并在判别器中使用残差连接，详情参见第4.1节。

针对数据集的调优与Karras等人[24]类似，我们在FFHQ数据集中增加了水平翻转操作，从而有效将训练图像的数量从7万增加到14万。对于LSUN数据集，则没有进行任何增强处理。我们发现，不同数据集和配置下的最优训练长度以及 R_1 正则化权重 γ 的选择往往差异较大。除了表1中的E配置以及表3中的LSUN CHURCH和LSUN HORSE外，所有训练运行均使用 $\gamma = 10$ ，在这些特定情况下我们采用 $\gamma = 100$ 。进一步调整 γ 可能还会带来额外的好处。

性能优化 我们对训练运行进行了详尽的剖析，发现——表达式简化为

在我们的案例中——默认用于图像过滤、上/下采样、偏差添加和泄漏ReLU的操作，在训练时间和GPU内存占用方面具有较高的开销。这促使我们使用手写的CUDA内核来优化这些操作。我们将滤波后的上/下采样实现为一个融合的单一操作，并将偏差和激活函数实现为另一个单一操作。在配置E下的1024²分辨率，我们的优化使整体训练时间提高了约30%，内存占用减少了约20%。

$$\begin{aligned}\mathcal{L}_{\mathbf{w}} &= \mathbb{E}_{\mathbf{y}} \left(\left\| \mathbf{U} \tilde{\Sigma} \mathbf{V}^T \mathbf{y} \right\|_2 - a \right)^2 \\ &= \mathbb{E}_{\mathbf{y}} \left(\left\| \tilde{\Sigma} \mathbf{y} \right\|_2 - a \right)^2.\end{aligned}$$

此外， $\tilde{\Sigma}$ 中的零矩阵会降低 $\mathbf{y}$ 在 L 维度之外的维度，从而有效地将其分布边缘化。边缘化的分布依然是剩余 L 维度上的单位正态分布。接下来我们需要考虑的是最小化该表达式的值。

C. 路径长度正则化的效应

第3.2节中描述的路径长度正则化项形式为：

$$\mathcal{L}_{\text{pl}} = \mathbb{E}_{\mathbf{w}} \mathbb{E}_{\mathbf{y}} \left(\left\| \mathbf{J}_{\mathbf{w}}^T \mathbf{y} \right\|_2 - a \right)^2, \quad (6)$$

其中， $\mathbf{y} \in \mathbb{R}^M$ 是生成图像空间（维度为 $M = 3wh$ ，即 RGB 图像尺寸）中的单位正态分布随机变量， $\mathbf{J}_{\mathbf{w}} \in \mathbb{R}^{M \times L}$ 是生成函数 $g: \mathbb{R}^L \mapsto \mathbb{R}^M$ 在隐空间点 $\mathbf{w} \in \mathbb{R}^L$ 处的雅可比矩阵，而 $a \in \mathbb{R}$ 是一个全局值，表示所需的梯度尺度。

C.1. 对逐点雅可比矩阵的影响

该先验值在每个潜在空间点 $\mathbf{w}$ 上分别使内部期望值最小化时达到最小。在此子部分中，我们展示了当雅可比矩阵 $\mathbf{J}_{\mathbf{w}}$ （在一个全局缩放因子范围内）正交时，内部期望值会（近似地）被最小化。一般策略是利用一个众所周知的事实，在高维空间 L 中，单位正态分布的密度集中在半径为 $\sqrt{L}$ 的球壳上。此时，矩阵 $\mathbf{J}_{\mathbf{w}}^T$ 将使预期函数在其极小值位于该半径处时内部期望值最小化。这可以通过任何具有适当全局缩放因子（在每个 $\mathbf{w}$ 中相同）的正交矩阵实现。

我们首先考虑内层期望值

$$\mathcal{L}_{\mathbf{w}} := \mathbb{E}_{\mathbf{y}} \left(\left\| \mathbf{J}_{\mathbf{w}}^T \mathbf{y} \right\|_2 - a \right)^2.$$

我们首先注意到， $\mathbf{y}$ 的径向对称分布以及 l_2 范数的分布特性允许我们将注意力集中在对角矩阵上。这一点可以通过奇异值分解（SVD）来观察，其中 $\mathbf{U} \in \mathbb{R}^{L \times L}$ 和 $\mathbf{V} \in \mathbb{R}^{M \times M}$ 是正交矩阵，而 $\tilde{\Sigma} = [\Sigma \mathbf{0}]$ 是由一个对角矩阵 $\Sigma \in \mathbb{R}^{L \times L}$ 与一个零矩阵 $\mathbf{0} \in \mathbb{R}^{L \times (M-L)}$ 水平拼接而成的组合[15]。由于用正交矩阵旋转单位标准随机变量不会改变其分布特性，并且旋转向量也不会改变它的

$$\mathcal{L}_{\mathbf{w}} = \mathbb{E}_{\tilde{\mathbf{y}}} \left(\left\| \Sigma \tilde{\mathbf{y}} \right\|_2 - a \right)^2,$$

对角线上的方阵 $\Sigma \in \mathbb{R}^{L \times L}$ ，其中 $\tilde{\mathbf{y}}$ 在维度 L 上服从单位正态分布。总之，所有与 Σ 具有相同奇异值的矩阵 $\mathbf{J}_{\mathbf{w}}^T$ 都会产生相同的原始损失值。

接下来，我们证明当对角矩阵 Σ 的每个对角元素具有相同的特定值时，即它是单位矩阵的一个常数倍时，该表达式取得最小值。首先，我们将期望表示为概率密度 $\tilde{\mathbf{y}}$ 的积分：

$$\begin{aligned}\mathcal{L}_{\mathbf{w}} &= \int \left(\left\| \Sigma \tilde{\mathbf{y}} \right\|_2 - a \right)^2 p_{\tilde{\mathbf{y}}}(\tilde{\mathbf{y}}) d\tilde{\mathbf{y}} \\ &= (2\pi)^{-\frac{L}{2}} \int \left(\left\| \Sigma \tilde{\mathbf{y}} \right\|_2 - a \right)^2 \exp \left(-\frac{\tilde{\mathbf{y}}^T \tilde{\mathbf{y}}}{2} \right) d\tilde{\mathbf{y}}\end{aligned}$$

观察密度的径向对称形式，我们转换到极坐标系 $\tilde{\mathbf{y}} = r\phi$ 中，在该坐标系中 $r \in \mathbb{R}_+$ 表示原点的距离，而 $\phi \in \mathbb{S}^{L-1}$ 是一个单位矢量，即位于一维单位球面上的一个点。这种变量变换引入了一个雅可比因子 r^{L-1} ：

$$\begin{aligned}\tilde{\mathcal{L}}_{\mathbf{w}} &= (2\pi)^{-\frac{L}{2}} \int_{\mathbb{S}} \int_0^\infty (r \left\| \Sigma \phi \right\|_2 - a)^2 r^{L-1} \\ &\quad \exp \left(-\frac{r^2}{2} \right) dr d\phi\end{aligned}$$

概率密度 $(2\pi)^{-L/2} r^{L-1} \exp \left(-\frac{r^2}{2} \right)$ 是一个以极坐标表示的 L 维单位正态分布，仅依赖于半径而不依赖角度。泰勒展开的标准论证表明，在 L 较高时，对于任意 ϕ ，该密度可以近似为 $(2\pi e/L)^{-L/2} \exp \left(-\frac{1}{2}(r - \mu)^2/\sigma^2 \right)$ 所表示的一维非归一化正态分布，其中心位于 $\mu = \sqrt{L}$ ，标准差为 $\sigma = 1/\sqrt{2}$ [4]。换句话说， L 维单位正态分布的密度集中在半径为 $\sqrt{L}$ 的壳层上。将此密度代入积分中，损失函数变为

approximately

$$\mathcal{L}_{\mathbf{w}} \approx (2\pi e/L)^{-L/2} \int_{\mathbb{S}} \int_0^\infty (r \|\Sigma \phi\|_2 - a)^2 \exp\left(-\frac{(r - \sqrt{L})^2}{2\sigma^2}\right) dr d\phi, \quad (7)$$

在维度趋于无穷大时，该近似变得精确。 L

为了最小化这一损失，我们将 Σ 设定为使得函数 $(r \|\Sigma \phi\|_2 - a)^2$ 在半径为 $\sqrt{L}$ 的球壳上取得极小值。通过 $\Sigma = \frac{a}{\sqrt{L}} \mathbf{I}$ ，该函数在 ϕ 上变得恒定，表达式简化为

$$\mathcal{L}_{\mathbf{w}} \approx (2\pi e/L)^{-L/2} \mathcal{A}(\mathbb{S}) a^2 L^{-1} \int_0^\infty (r - \sqrt{L})^2 \exp\left(-\frac{(r - \sqrt{L})^2}{2\sigma^2}\right) dr,$$

其中 $\mathcal{A}(\mathbb{S})$ 是单位球的表面积（和其他常数因子一样，与最小化无关）。需要注意的是，抛物线的零点 $(r - \sqrt{L})^2$ 与概率密度的最大值重合，因此选择的 Σ 可以使方程7中的内积分对于每一个 ϕ 单独达到最小。

总之，我们证明了——假设潜在空间的高维性——生成器雅可比矩阵的所有奇异值在每个潜在空间点上都等于一个全局常数时，路径长度先验（公式6）的值被最小化，即它们在一个全局恒定的比例下正交。

在理论上， a 仅调整映射的值而不改变其属性，并可以设置为固定值（例如，1）。然而，在实践中，它确实会影响训练的动力学。如果施加的比例与网络随机初始化所诱导的比例不匹配，则训练会在关键的早期步骤中将权重推向所需的总体幅度，而不是执行实际感兴趣的优化目标。这可能会恶化网络权重的内部状态，并导致后续训练中的次优性能。经验表明，设置固定比例会降低不同训练运行和数据集之间结果的一致性。相反，我们根据雅可比矩阵现有尺度的滑动平均值动态地调整 a ，即 $a \approx \mathbb{E}_{\mathbf{w}, \mathbf{y}} (\|\mathbf{J}_{\mathbf{w}}^T \mathbf{y}\|_2)$ 。通过这种选择，先验目标将局部雅可比矩阵的比例朝向已存在的全局平均值进行调整，而不是强制特定的全局平均值。这也消除了测量雅可比矩阵适当比例的需求。

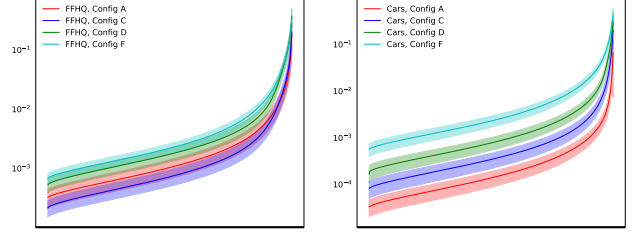


图17. 在随机潜在空间点 $\mathbf{w}$ 处计算的雅可比矩阵排序奇异值的均值和标准差，其中最大特征值被归一化为1。在两个数据集中，路径长度正则化（配置D）和新架构（配置F）表现出更好的条件数；特别地，在包含更多变异性的小汽车数据集上，路径长度正则化的效果对PPL指标的影响更为显著（见表1）。

明确地进行处理，如 Odena 等人 [33] 所做的那样，他们考虑了一个相关的条件先验。

图17展示了使用和不使用路径长度正则化的网络训练过程中雅可比矩阵奇异值的实测幅度。虽然未达到完全正交，但经过正则化处理的网络的特征值彼此更接近，表明条件更好，其效果强度与PPL指标（表1）相关。

C.2. 对生成器映射全局属性的影响

在前一子节中，我们发现先验鼓励生成器映射的雅可比矩阵处处正交。虽然图17显示该映射实际上并不完全满足这一约束条件，但考虑对于确实满足此约束条件的映射所具有的全局性质是有启发性的。不失一般性，为简化表述，假设矩阵具有单位全局尺度。

关键性质在于映射 $g: \mathbb{R}^L \mapsto \mathbb{R}^M$ 处处具有正交雅可比矩阵，从而保持曲线的长度。为了说明这一点，设 $u: [t_0, t_1] \mapsto \mathbb{R}^L$ 参数化潜在空间中的某条曲线。通过生成器 g 对该曲线进行映射后，我们得到图像空间中的一条曲线 $\tilde{u} = g \circ u$ 。其弧长为

$$L = \int_{t_0}^{t_1} |\tilde{u}'(t)| dt, \quad (8)$$

其中的 prime 表示对 t 求导。根据链式法则，这等于

$$L = \int_{t_0}^{t_1} |J_g(u(t)) u'(t)| dt, \quad (9)$$

其中 $J_g \in \mathbb{R}^{L \times M}$ 是 g 在 $u(t)$ 处的雅可比矩阵。根据我们的假设，该雅可比矩阵是正交的，并且

因此，它不影响向量 $u'(t)$ 的2范数：

$$L = \int_{t_0}^{t_1} |u'(t)| dt. \quad (10)$$

这是曲线 u 在潜在空间中的长度，在映射到 g 之前。因此， u 和 $\tilde{u}$ 的长度相等，所以 g 保持任何曲线的长度不变。

在微分几何的语言中， g 将欧几里得潜在空间 $\mathbb{R}^L$ 等距嵌入到流形 $\mathcal{M}$ 中——例如，在所有可能的RGB图像的空间内嵌入表示人脸的流形。等距的一个结果是，潜在空间中的直线段映射为流形上的测地线（即最短路径）：连接两个潜在空间点的直线 v 无法变得更短，因此在对应的图像之间也无法存在比 $g \circ v$ 更短的流形上图像空间路径。例如，在人脸图像流形上的测地线是两个面部之间的连续变形，该变形导致最小总变化量（以RGB空间中的 l_2 差异来衡量），当累加每个变形步骤中的图像差值时。

在实际操作中，等距性并未通过之前的实验证明实现。训练的完整损失函数是由可能存在冲突的标准组合而成的，并且尚不清楚一个真正意义上的等距映射是否能够表达出感兴趣的图像流形。尽管如此，尽可能使映射接近等距的压力会产生一些积极的结果。具体而言，这会抑制不必要的“绕路”：在一个非约束生成器映射中，两个相似图像之间的潜在空间插值可能经过RGB空间中的任意数量的遥远图像。通过正则化，映射被鼓励将遥远的图像放置在潜在空间的不同区域，从而获得任何两个端点之间较短的图像路径。

D. 投影方法细节

给定目标图像 x ，我们寻求找到对应的 $w \in \mathcal{W}$ 和每层噪声图，记为 $n_i \in \mathbb{R}^{r_i \times r_i}$ ，其中 i 表示层数索引， r_i 表示第 i 个噪声图的分辨率。在 1024×1024 分辨率下的基线 StyleGAN 生成器有 18 个噪声输入，即对于从 4×4 到 1024×1024 像素的每个分辨率都有两个。我们的改进架构少了一个噪声输入，因为我们没有向学习到的 4×4 常量添加噪声（见图 2）。

在优化之前，我们通过运行映射网络来计算 $\mu_w = \mathbb{E}_z f(z)$ ，具体方法是使用 10,000 个随机潜在代码 z 。我们还通过计算 $\sigma_w^2 = \mathbb{E}_z \|f(z) - \mu_w\|_2^2$ （即到中心的平均平方欧氏距离）来近似估计 $\mathcal{W}$ 的规模。

在优化开始时，我们为所有 i 初始化 $w = \mu_w$ 和 $n_i = \mathcal{N}(0, \mathbf{I})$ 。可训练参数为

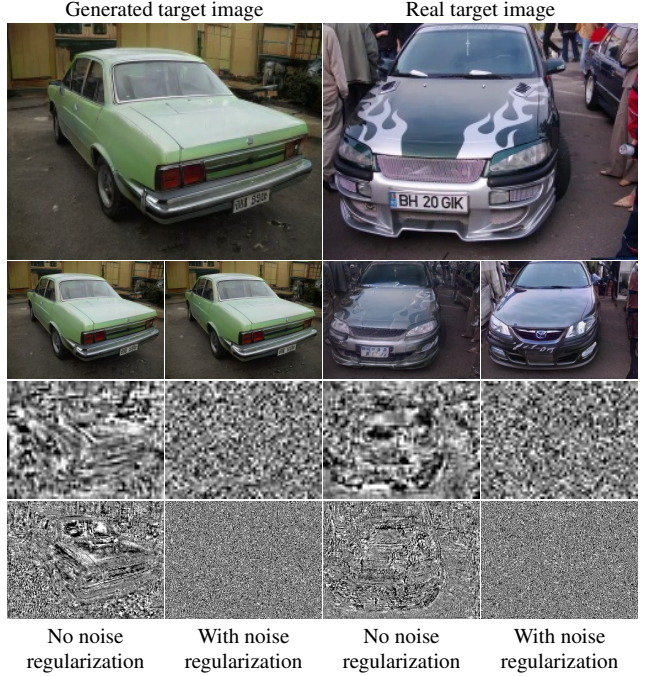


图18. 隐空间投影中噪声正则化的效果，在此过程中我们还优化了合成网络的噪声输入内容。从上到下依次为：目标图像、重新合成的图像以及不同分辨率下的两个噪声图的内容。当关闭噪声正则化时，我们仅将噪声图归一化至零均值和单位方差，这会导致优化过程将信号渗入噪声图中。启用噪声正则化可以防止这种情况发生。此处使用的模型对应表1中的配置F。

$\{v_{33}\}$ 及其所有噪声图 $\{v_{34}\}$ 中的组件。优化过程运行1000次迭代，使用默认参数的Adam优化器[25]。最大学习率为 $\{v_{35}\}0\{v_{36}\}1$ ，在前50次迭代中从零线性增加，并在最后250次迭代中通过余弦调度逐渐降至零。在优化过程的前三 quarter 中，在评估损失函数时向 $\{v_{37}\}$ 添加高斯噪声，其标准差为 $\{v_{38}\}0\{v_{39}\}0\{v_{40}\}05\{v_{41}\}$ ，其中 $\{v_{42}\}$ 在前750次迭代中从1线性减少到0。这增加了优化过程的随机性，并有助于稳定全局最优解的寻找。

鉴于我们明确地优化噪声图，我们必须小心避免实际信号渗入其中。因此，在损失函数中除了图像质量项外，还加入了若干噪声图正则化项。图像质量项是目标图像 x 和合成图像 $L_{image} = D_{LPIPS}[x, g(\tilde{w}, n_0, n_1, \dots)]$ 之间的 LPIPS [50] 距离。为了提高性能和稳定性，在计算 LPIPS 距离之前将两张图像均下采样至 256×256 分辨率。噪声图的正则化操作在

	SN-G	SN-D	Demod	P.reg	FID ↓	PPL ↓	Pre. ↑	Rec. ↑
1	—	—	✓	✓	2.83	145.0	0.689	0.492
2	—	✓	✓	✓	2.98	131.4	0.700	0.469
3	✓	✓	✓	✓	3.40	130.9	0.720	0.435
4	✓	✓	—	✓	3.38	162.6	0.705	0.468
5	✓	✓	—	—	3.33	394.9	0.705	0.463
6	✓	—	—	✓	3.36	217.1	0.695	0.464
7	✓	—	—	—	3.22	394.4	0.692	0.489

表4. 光谱归一化对FFHQ (1024分辨率) 的影响。第一行对应于StyleGAN2, 即表1中的配置F。在后续的各行中, 我们在生成器中启用光谱归一化 (SN-G), 并在判别器中启用光谱归一化 (SN-D)。我们还测试了不使用权重调制训练的情况 (Demod) 以及路径长度正则化的缺失情况 (P.reg)。所有这些配置对FID和Recall指标都有很大的负面影响。↑表示数值越高越好, 而↓表示数值越低越好。

多个分辨率尺度。为此, 我们为每个大于8×8大小的噪声图形成一个向下采样到8×8分辨率的金字塔, 通过平均2×2像素邻域并在每一步乘以2来保持预期的单位方差。这些下采样的噪声图仅用于正则化, 不参与合成过程。

令原始噪声图为 $\mathbf{n}_{i,0} = \mathbf{n}_i$, 下采样后的版本为 $\mathbf{n}_{i,j>0}$ 。类似地, 设 $r_{i,j}$ 为原始 ($j = 0$) 或下采样 ($j > 0$) 噪声图的分辨率, 使得 $r_{i,j+1} = r_{i,j}/2$ 成立。对于噪声图 $\mathbf{n}_{i,j}$ 的正则化项则是

$$L_{i,j} = \left(\frac{1}{r_{i,j}^2} \cdot \sum_{x,y} \mathbf{n}_{i,j}(x,y) \cdot \mathbf{n}_{i,j}(x-1,y) \right)^2 + \left(\frac{1}{r_{i,j}^2} \cdot \sum_{x,y} \mathbf{n}_{i,j}(x,y) \cdot \mathbf{n}_{i,j}(x,y-1) \right)^2,$$

噪声图在边缘处被视为循环。因此, 正则化项是分辨率归一化自相关系数在一像素水平和垂直偏移下的平方和之和, 在正常分布信号的情况下该值应为零。总的损失函数即为 $L_{total} = L_{image} + \alpha \sum_{i,j} L_{i,j}$ 。在所有测试中, 我们使用的噪声正则化权重均为 $\alpha = 10^5$ 。此外, 我们在每次优化步骤后将所有噪声图重新归一化至均值为零、方差为单位的分布。图18展示了噪声正则化对最终噪声图的影响。

附录E 带谱范数的实验结果

由于光谱归一化 (SN) 在生成对抗网络 (GANs) 中被广泛使用, 我们研究了它对StyleGAN2的影响。表4给出了在启用光谱归一化的情况下, 结合或替代我们的技术 (权重解调、路径长度正则化) 的各种配置的结果。

Item	GPU years (Volta)	Electricity (MWh)
Initial exploration	20.25	58.94
Paper exploration	13.71	31.49
FFHQ config F	0.23	0.68
Other runs in paper	7.20	16.77
Backup runs left out	4.73	12.08
Video, figures, etc.	0.31	0.82
Public release	4.62	10.82
Total	51.05	131.61

表5. 该项目的计算工作量和电力消耗数据。计算单位为单个NVIDIA V100 GPU年的GPU时——使用单一GPU执行此项目大约需要51年时间。有关计算和能耗估算的更多细节, 请参见正文。*Initial exploration* 包括自StyleGAN [24] 发布以来影响我们启动该项目的训练运行。*Paper exploration* 包括专门为该项目进行但未打算直接用于论文中的所有训练运行。*FFHQ config F* 指最终网络的训练过程, 这大约相当于在未经超参数调整的情况下为另一个数据集训练网络的成本。*Other runs in paper* 覆盖了论文中展示的所有其他网络的训练情况。*Backup runs left out* 包括可能被纳入论文但最终为了使论述更加聚焦而未包含的各种网络的训练过程。*Video, figures, etc.* 包括用于生成论文中的图像和图表以及结果视频所花费的计算量。*Public release* 覆盖了与公共发布相关的测试、基准测试及大规模图像导出工作。

有趣的是, 将谱范数规范化添加到生成器中几乎没有任何影响。在实现层面, SN通过一个标量值 $1/\sigma(w)$ 对每一层的权重张量进行缩放。然而, 这种缩放的效果会被主卷积层以及仿射变换层中的公式3所覆盖。因此, SN仅通过其对tRGB层的影响而添加到权重解调之上。

启用光谱归一化后, 判别器的FID指标略有下降。即使其影响仅限于tRGB层, 在生成器中也同时启用它会导致结果显著变差。部分保留光谱归一化的同时禁用我们的一些贡献也没有改善情况。因此, 我们可以得出结论, StyleGAN2在不使用光谱归一化的情况下能取得更好的效果。

F. 能耗

计算是任何机器学习项目中的核心资源: 其可用性和成本, 以及相关的能耗, 都是选择研究方向和实际应用的关键因素。我们在表5中详细列出了整个项目的GPU时间和电力消耗情况。

我们报告的计算工作量以单张GPU年的形式表示 (伏尔塔级GPU)。我们使用了不同数量的

使用 NVIDIA DGX-1s 设备进行项目的不同阶段，并通过简单地按使用的 GPU 数量缩放，将每次运行转换为单 GPU 对等项。

该项目总计消耗了约131.61兆瓦时 (MWh) 的电力。我们遵循了Green500功率测量指南[11]，具体做法如下：对于每个任务，我们记录了其确切运行时间、使用的GPU数量以及该任务是在我们的两个独立计算集群中的哪一个上执行的。然后，我们在训练FFHQ配置F时测量了一个配备8个GPU的DGX-1的实际功耗。由于这两个集群使用不同的DGX-1型号，因此分别对其进行了估算。我们绝大多数的训练运行都使用了8个GPU，对于其他情况，则通过线性缩放 $n/8$ 来近似计算功耗，其中 n 表示使用的GPU数量。

总计能源消耗的大约一半用于早期探索和形成想法。随后，四分之一的的时间用于通过更针对性的实验来完善这些想法，最后四分之一的的时间则用于撰写本文并准备代码、训练模型以及大量图像集的公开发布。训练单个FFHQ网络（配置F）大约耗电0.68兆瓦时（占整个项目支出的0.5%）。这是从头开始训练该网络的成本，可能使用不同的数据集。简而言之，绝大部分电力用于塑造想法、测试假设和调整超参数。我们没有使用自动化工具来寻找超参数或优化网络架构。